PressurePause Reclaimer

PSI-Gated Direct Reclaim Coordination

CS325 Semester Project (CLO 2) — Linux Kernel Modification

Course: CS325 — Operating Systems
Instructor: Dr. Mohammad Imran
Lab Instructor: Mam Areaba Fatime
CLO: CLO 2
Student: Huzaifa Huzaifa
Reg. ID: 241110
Kernel tree: Linux v6.8.12 (`LOCALVERSION=-baseline`)

Contents

1. Introduction

When physical memory is exhausted, allocating threads can enter **direct reclaim**: they block while the kernel evicts pages synchronously. Under coordinated memory thrash, many threads reclaim in parallel, evicting pages that are immediately refaulted. The working set exceeds available RAM, swap I/O rises, and tail latency degrades.

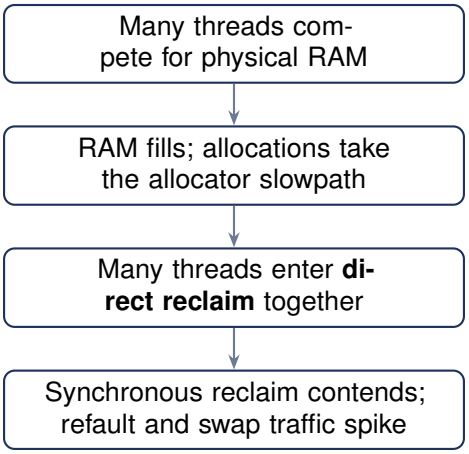


Figure 1. *Conceptual chain from contention to reclaim storms (motivation for coordinating direct reclaim).*

PressurePause is a kernel modification that adds a bounded, PSI-aware coordination step on the direct reclaim path. When memory PSI `some` avg10 exceeds 0.01% (basis-point compare against the same fixed-point values as `/proc/pressure/memory`), the patch wakes `kswapd` explicitly and applies a short, capped wait for background progress, then always runs the stock `shrink_zones()` priority loop. The design never disables reclaim permanently and always preserves stock behavior on timeout or when guards fire.

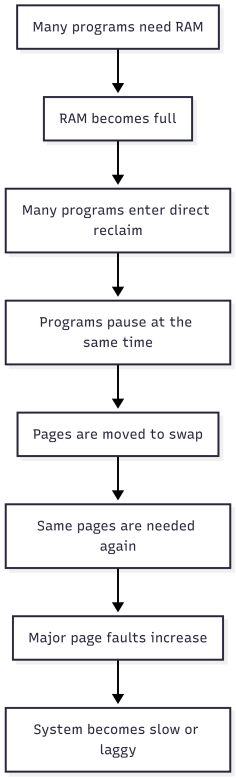


Figure 2. *Host under heavy memory pressure during a memory-intensive workload (elevated swap use and reclaim-related activity). This motivates coordinating foreground direct reclaim with background reclaim.*

This report covers the development environment and baseline build (Sections 2–3), component identification (Section 4), design and patch implementation (Sections 5–6), correctness properties (Section 7), and evaluation evidence that the new code path runs (Section 8).

2. Environment

- **Host:** Ubuntu 22.04 (Jammy), x86_64
- **Fallback kernel:** `6.8.0-117-generic` (distribution build)
- **Custom baseline:** `6.8.12-baseline` (vanilla stable v6.8.12, unpatched)
- **Source:** vendored Linux v6.8.12 at `kernel/linux`; tag recorded in `docs/kernel-tag.txt`.
- **Patch export:** `kernel/patches/0001-pressurepause.patch`
- **Configuration:** `make defconfig`; saved as `kernel/configs/config-6.8.12-baseline`
- **Build:** `make -j2 LOCALVERSION=-baseline`
- **Tools:** GCC toolchain, `dwarves`, `cscope`, `stress-ng`, `linux-tools-$(uname -r)`

Defconfig was chosen instead of copying the Ubuntu `/boot/config-*` file to avoid missing certificate paths (e.g. `SYSTEM_TRUSTED_KEYS`) that broke an earlier vanilla build attempt.

3. Kernel Build and Boot

This section documents the steps to obtain a bootable custom baseline kernel. Terminal screenshots are included for each major step.

3.1 Host Preparation

Build dependencies were installed with `apt`:

Install build dependencies

```
sudo apt update
sudo apt install -y build-essential libncurses-dev bison flex libssl-dev \
  libelf-dev bc dwarves git cscope stress-ng linux-tools-common \
  linux-tools-$(uname -r)
```

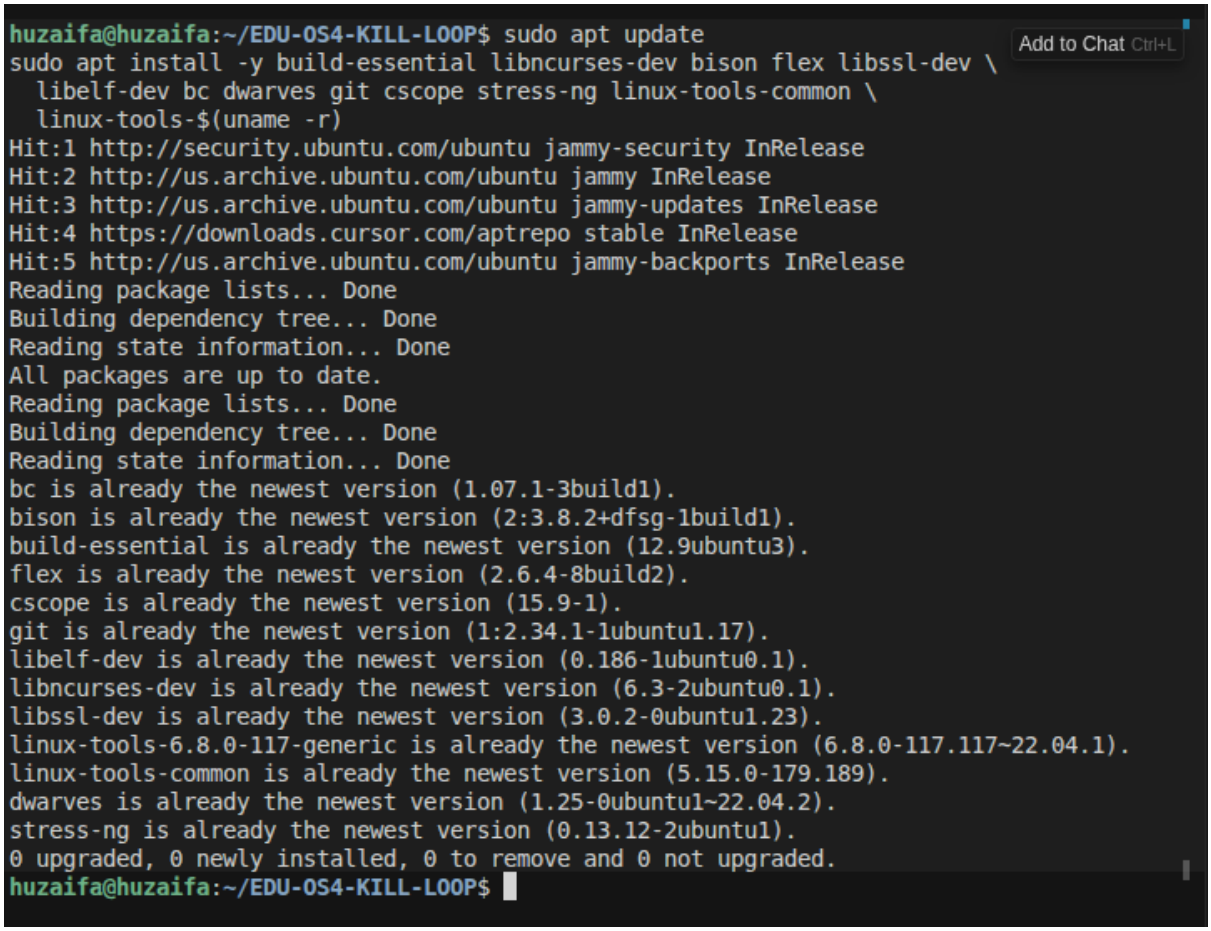


Figure 3. *Installing kernel build and benchmark packages; all packages were already present on the host.*

3.2 Source Tree

The kernel tree is vendored in this project repository at `kernel/linux`; a fresh checkout is `git clone` of the project, not a nested submodule. During initial setup the stable tree was cloned shallowly on branch `v6.8`, then checked out at tag `v6.8.12`:

Clone and pin kernel tag

```
cd /home/huzaifa/EDU-OS4-KILL-LOOP
mkdir -p kernel docs kernel/configs kernel/patches
git clone --depth=1 --branch v6.8 \
  https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git kernel/linux
cd kernel/linux
git fetch --depth=1 origin tag v6.8.12
git checkout v6.8.12
git describe --always > ../../docs/kernel-tag.txt
```

```
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP$ cd /home/huzaifa/EDU-OS4-KILL-LOOP
mkdir -p kernel docs kernel/configs kernel/patches
git clone --depth=1 --branch v6.8 https://git.kernel.org/pub/scm/linux/kernel/git/stabl
e/linux.git kernel/linux
cd kernel/linux
git fetch --depth=1 origin tag v6.8.12
git checkout v6.8.12
git describe --always > ../../docs/kernel-tag.txt
Cloning into 'kernel/linux'...
remote: Enumerating objects: 88457, done.
remote: Counting objects: 100% (88457/88457), done.
remote: Compressing objects: 100% (86182/86182), done.
remote: Total 88457 (delta 6727), reused 18232 (delta 1354), pack-reused 0 (from 0)
Receiving objects: 100% (88457/88457), 246.16 MiB | 3.12 MiB/s, done.
Resolving deltas: 100% (6727/6727), done.
Note: switching to 'e8f897f4afef0031fe618a8e94127a0934896aba'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

Updating files: 100% (83475/83475), done.
remote: Enumerating objects: 7498, done.
remote: Counting objects: 100% (7498/7498), done.
remote: Compressing objects: 100% (2651/2651), done.
remote: Total 3775 (delta 3636), reused 1226 (delta 1117), pack-reused 0 (from 0)
Receiving objects: 100% (3775/3775), 852.39 KiB | 3.17 MiB/s, done.
Resolving deltas: 100% (3636/3636), completed with 3625 local objects.
From https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux
 * [new tag]             v6.8.12    -> v6.8.12
Previous HEAD position was e8f897f4a Linux 6.8
HEAD is now at 632428373 Linux 6.8.12
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$
```

Figure 4. *Cloning the stable tree and checking out tag v6.8.12 (`HEAD` at commit for Linux 6.8.12).*

3.3 Kernel Configuration

A default x86_64 configuration was generated and saved for reproducibility:

defconfig

```
cd kernel/linux
make defconfig
make olddefconfig
cp .config ../configs/config-6.8.12-baseline
```

```
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$ cd /home/huzaifa/EDU-OS4-KILL-L
OOP/kernel/linux
make defconfig
make olddefconfig
cp .config ../configs/config-6.8.12-baseline
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/kconfig/conf.o
  HOSTCC  scripts/kconfig/confdata.o
  HOSTCC  scripts/kconfig/expr.o
  LEX     scripts/kconfig/lexer.lex.c
  YACC    scripts/kconfig/parser.tab.[ch]
  HOSTCC  scripts/kconfig/lexer.lex.o
  HOSTCC  scripts/kconfig/menu.o
  HOSTCC  scripts/kconfig/parser.tab.o
  HOSTCC  scripts/kconfig/preprocess.o
  HOSTCC  scripts/kconfig/symbol.o
  HOSTCC  scripts/kconfig/util.o
  HOSTLD  scripts/kconfig/conf
*** Default configuration is based on 'x86_64_defconfig'
#
# configuration written to .config
#
#
# No change to .config
#
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$
```

Figure 5. *`make defconfig` based on `x86_64_defconfig`; configuration written to `.config` and copied to `kernel/configs/`.*

3.4 Compilation

The kernel was built with a distinct `LOCALVERSION` suffix so it does not collide with distribution packages:

Build baseline kernel

```
make -j2 LOCALVERSION=-baseline 2>&1 | tee ~/build.log
```

```
  CC      arch/x86/boot/version.o
  CC      arch/x86/boot/video-vga.o
  CC      arch/x86/boot/video-vesa.o
  CC      arch/x86/boot/video-bios.o
  HOSTCC  arch/x86/boot/tools/build
  CC      arch/x86/boot/compressed/error.o
  CPUSTR  arch/x86/boot/cpustr.h
  CC      arch/x86/boot/cpu.o
  OBJCOPY arch/x86/boot/compressed/vmlinux.bin
  HOSTCC  arch/x86/boot/compressed/mkpiggy
  CC      arch/x86/boot/compressed/cpuflags.o
  CC      arch/x86/boot/compressed/early_serial_console.o
  CC      arch/x86/boot/compressed/kaslr.o
  CC      arch/x86/boot/compressed/ident_map_64.o
  CC      arch/x86/boot/compressed/idt_64.o
  AS      arch/x86/boot/compressed/idt_handlers_64.o
  CC      arch/x86/boot/compressed/pgtable_64.o
  CC      arch/x86/boot/compressed/acpi.o
  CC      arch/x86/boot/compressed/efi.o
  AS      arch/x86/boot/compressed/efi_mixed.o
  CC      arch/x86/boot/compressed/misc.o
  GZIP    arch/x86/boot/compressed/vmlinux.bin.gz
  MKPIGGY arch/x86/boot/compressed/piggy.S
  AS      arch/x86/boot/compressed/piggy.o
  LD      arch/x86/boot/compressed/vmlinux
  ZOFFSET arch/x86/boot/zoffset.h
  OBJCOPY arch/x86/boot/vmlinux.bin
  AS      arch/x86/boot/header.o
  LD      arch/x86/boot/setup.elf
  OBJCOPY arch/x86/boot/setup.bin
  BUILD   arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready  (#1)
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$
```

Figure 6. *Successful build: `arch/x86/boot/bzImage` is ready for installation.*

3.5 Installation

After a successful `make`, modules and the boot image were installed:

Install kernel

```
sudo make modules_install install
```

```
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$ sudo make modules_install install
sudo update-grub
ls /boot/vmlinuz*
[sudo] password for huzaifa:
  SYMLINK /lib/modules/6.8.12-baseline/build
  INSTALL /lib/modules/6.8.12-baseline/modules.order
  INSTALL /lib/modules/6.8.12-baseline/modules.builtin
  INSTALL /lib/modules/6.8.12-baseline/modules.builtin.modinfo
  INSTALL /lib/modules/6.8.12-baseline/kernel/fs/efivarfs/efivarfs.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/nf_log_syslog.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_mark.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_nat.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_LOG.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_MASQUERADE.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_addrtype.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/ipv4/netfilter/iptable_nat.ko
  DEPMOD  /lib/modules/6.8.12-baseline
  INSTALL /boot
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
update-initramfs: Generating /boot/initrd.img-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
I: /boot/initrd.img.old is now a symlink to initrd.img-6.8.0-117-generic
I: /boot/initrd.img is now a symlink to initrd.img-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/zz-shim 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.12-baseline
Found initrd image: /boot/initrd.img-6.8.12-baseline
Found linux image: /boot/vmlinuz-6.8.0-117-generic
Found initrd image: /boot/initrd.img-6.8.0-117-generic
Found linux image: /boot/vmlinuz-6.8.0-40-generic
Found initrd image: /boot/initrd.img-6.8.0-40-generic
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done
```

Figure 7. *`modules_install` and `install` placing `vmlinuz-6.8.12-baseline` and modules under `/lib/modules/6.8.12-baseline/`.*

3.6 Bootloader Update

GRUB was regenerated so the new kernel appears in the boot menu:

Update GRUB

```
sudo update-grub
ls /boot/vmlinuz*
```

```
Found linux image: /boot/vmlinuz-6.8.0-40-generic
Found initrd image: /boot/initrd.img-6.8.0-40-generic
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.12-baseline
Found initrd image: /boot/initrd.img-6.8.12-baseline
Found linux image: /boot/vmlinuz-6.8.0-117-generic
Found initrd image: /boot/initrd.img-6.8.0-117-generic
Found linux image: /boot/vmlinuz-6.8.0-40-generic
Found initrd image: /boot/initrd.img-6.8.0-40-generic
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done
/boot/vmlinuz                    /boot/vmlinuz-6.8.12-baseline
/boot/vmlinuz-6.8.0-117-generic  /boot/vmlinuz.old
/boot/vmlinuz-6.8.0-40-generic
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$
```

Figure 8. *update-grub detected /boot/vmlinuz-6.8.12-baseline alongside Ubuntu 6.8.0-117-generic and 6.8.0-40-generic.*

3.7 Boot Verification

The GRUB boot menu now lists the custom baseline kernel as the first selectable entry, while the stock Ubuntu kernels remain available as fallback options.

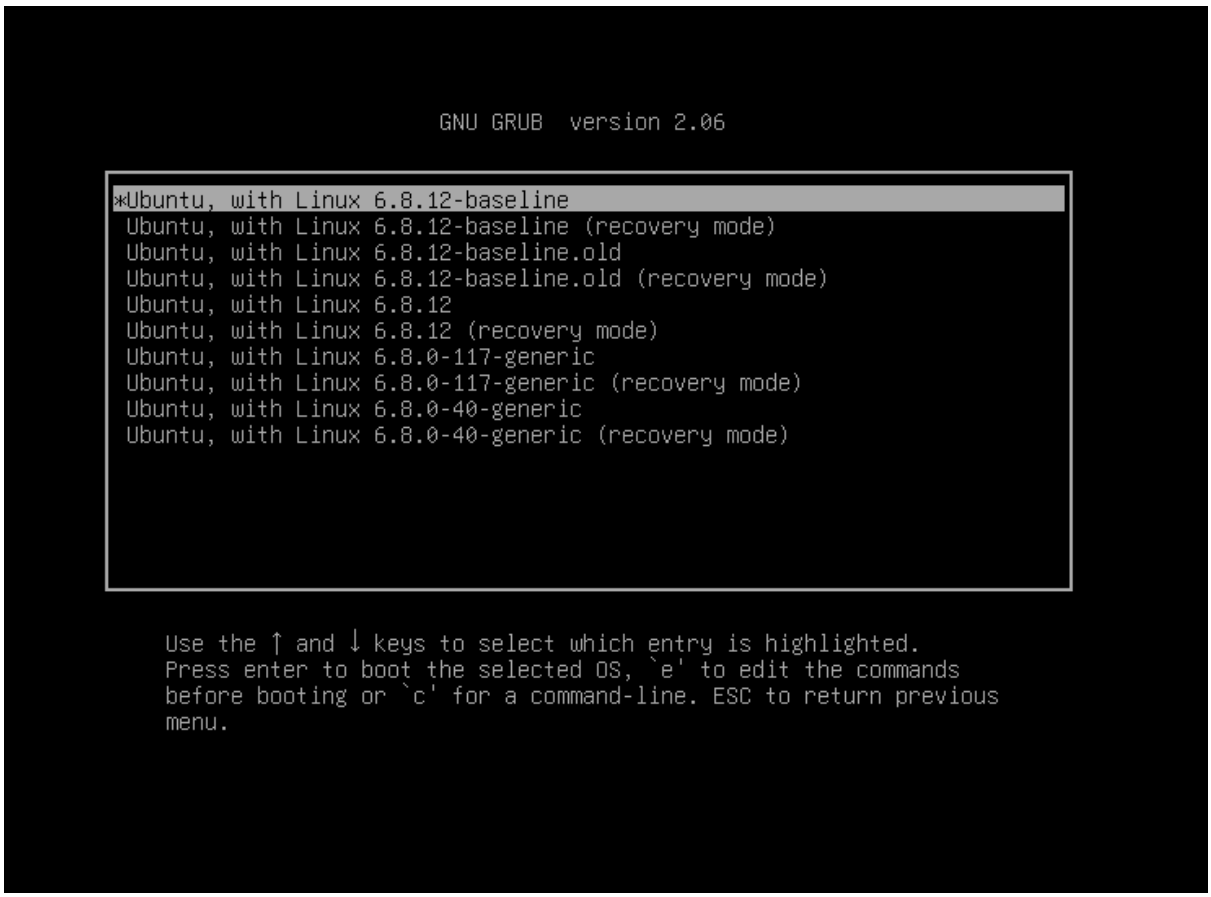


Figure 9. *GRUB menu showing `Ubuntu, with Linux 6.8.12-baseline` selected, with recovery and fallback Ubuntu kernel entries still available.*

Baseline install verified:

- boot image `/boot/vmlinuz-6.8.12-baseline`
- modules directory `/lib/modules/6.8.12-baseline/`
- GRUB entry present, then `uname -r` confirms `6.8.12-baseline` before Phase 4 benchmarks

3.8 Running kernels: baseline vs PressurePause

The custom baseline kernel and the PressurePause kernel both boot on the same host; the screenshots below show `uname -r` (and related shell context) for each installed image.

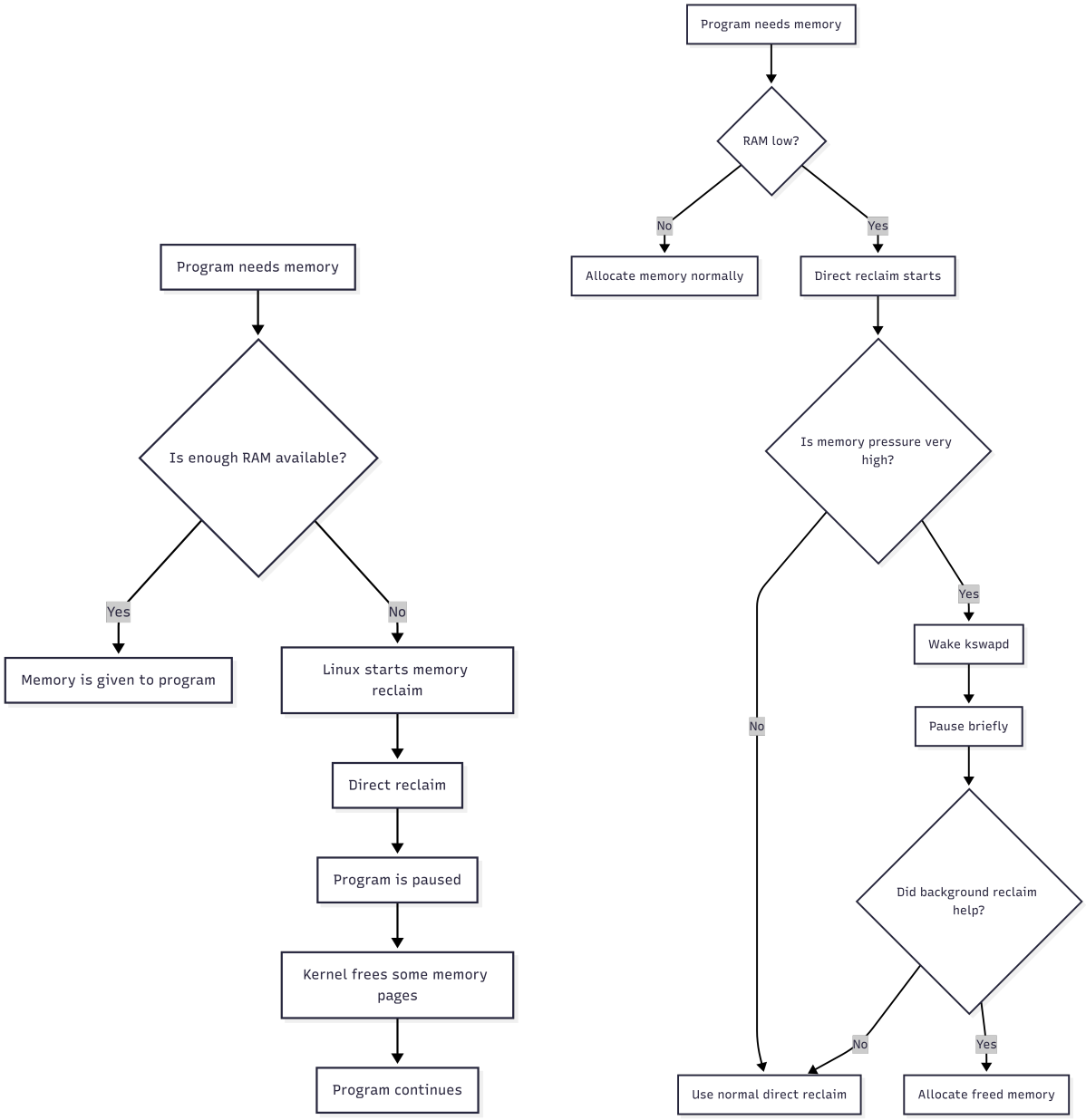


Figure 10. ***Left:*** *baseline / unpatched custom build (`6.8.12-baseline`).* ***Right:*** *patched PressurePause build (`6.8.12-pressurepause` or `6.8.12-pressurepause+` depending on `LOCALVERSION`).*

4. Component Identification

This section satisfies the course requirement to identify the selected kernel component within the source tree. The analysis targets Linux **v6.8.12** in `kernel/linux`. A markdown companion is maintained at `docs/identification.md`.

4.1 Selected Component

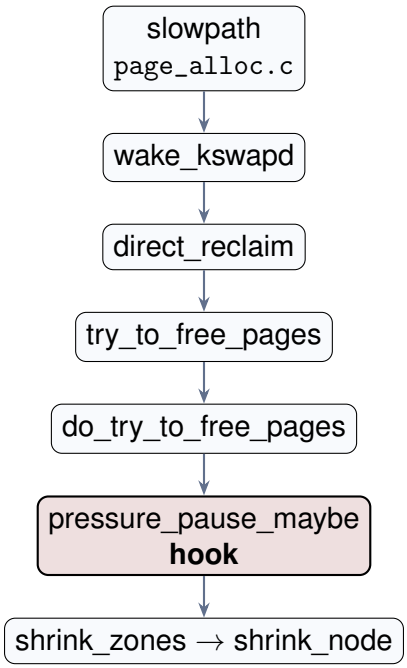
- **Course category:** Page replacement / reclaim (PressurePause extends direct reclaim coordination).

- **Kernel source files modified by PressurePause:** `include/linux/psi.h`, `kernel/sched/psi.c`, and `mm/vmscan.c`.
- **Related upstream (not modified):** `mm/page_alloc.c` — allocator slowpath and `__perform_reclaim` call into `try_to_free_pages()` in `mm/vmscan.c`; no PressurePause edits.
- **PSI background (mostly unchanged):** memory stall enums remain in `include/linux/psi_types.h`; accounting remains in `kernel/sched/psi.c`.
- **Implementation hook:** `pressure_pause_maybe()` is called from `do_try_to_free_pages()` in `mm/vmscan.c`, just before `shrink_zones()`.

4.2 Call Graph: Allocation to Reclaim

When `get_page_from_freelist()` cannot satisfy an allocation (zone free pages below watermarks), control enters the allocator slowpath and eventually direct reclaim:

1. `__alloc_pages_slowpath` — `mm/page_alloc.c` ~4040
2. `wake_all_kswapds` — `mm/page_alloc.c` ~3814 (background reclaim hint)
3. `__alloc_pages_direct_reclaim` — `mm/page_alloc.c` ~3781
4. `psi_memstall_enter` / `__perform_reclaim` — `mm/page_alloc.c` ~3755–3790
5. `try_to_free_pages` — `mm/vmscan.c` ~6604
6. `do_try_to_free_pages` — `mm/vmscan.c` ~6384; `pressure_pause_maybe()` at ~6397
7. `shrink_zones` → `shrink_node` — `mm/vmscan.c` (shared reclaim core)



4.3 Key Code Excerpts

Upstream (unchanged): direct reclaim entry (`mm/page_alloc.c`):

mm/page_alloc.c: __perform_reclaim (lines 3753–3770) — not modified by PressurePause

```
/* Perform direct synchronous page reclaim */
static unsigned long
__perform_reclaim(gfp_t gfp_mask, unsigned int order,
                  const struct alloc_context *ac)
{
    /* ... reclaim setup and compression retries ... */
    progress = try_to_free_pages(ac->zonelist, order, gfp_mask,
                                 ac->nodemask);
    /* ... */
    return progress;
}
```

Public direct-reclaim wrapper (`mm/vmscan.c`, stock function; line numbers from patched tree):

mm/vmscan.c: try_to_free_pages (lines 6604–6645)

```
unsigned long try_to_free_pages(struct zonelist *zonelist, int order,
                gfp_t gfp_mask, nodemask_t *nodemask)
{
    unsigned long nr_reclaimed;
    struct scan_control sc = {
        .nr_to_reclaim = SWAP_CLUSTER_MAX,
        .gfp_mask = current_gfp_context(gfp_mask),
        .reclaim_idx = gfp_zone(gfp_mask),
        .order = order,
        .nodemask = nodemask,
        .priority = DEF_PRIORITY,
        .may_writepage = !laptop_mode,
        .may_unmap = 1,
        .may_swap = 1,
    };

    BUILD_BUG_ON(MAX_PAGE_ORDER >= S8_MAX);
    BUILD_BUG_ON(DEF_PRIORITY > S8_MAX);
    BUILD_BUG_ON(MAX_NR_ZONES > S8_MAX);

    if (throttle_direct_reclaim(sc.gfp_mask, zonelist, nodemask))
        return 1;

    set_task_reclaim_state(current, &sc.reclaim_state);
    trace_mm_vmscan_direct_reclaim_begin(order, sc.gfp_mask);

    nr_reclaimed = do_try_to_free_pages(zonelist, &sc);

    trace_mm_vmscan_direct_reclaim_end(nr_reclaimed);
    set_task_reclaim_state(current, NULL);

    return nr_reclaimed;
}
```

PressurePause hook call site (mm/vmscan.c):

mm/vmscan.c: do_try_to_free_pages (lines 6394–6404)

```
1       if (!cgroup_reclaim(sc))
2           __count_zid_vm_events(ALLOCSTALL, sc->reclaim_idx, 1);
3
4       pressure_pause_maybe(zonelist, sc);
5
6       do {
7           if (!sc->proactive)
8               vmpressure_prio(sc->gfp_mask, sc->target_mem_cgroup,
9                       sc->priority);
10          sc->nr_scanned = 0;
11          shrink_zones(zonelist, sc);
```

4.4 Watermarks

Reclaim is triggered when a zone's free page count falls below configured **min**, **low**, or **high** watermarks during allocation. The fast path in `get_page_from_freelist()` fails; `__alloc_pages_slowpath()` may wake `kswapd`, attempt compaction, and finally call `__alloc_pages_direct_reclaim()` so the allocating thread performs synchronous reclaim.

4.5 kswapd vs. Direct Reclaim

Both paths converge on `shrink_node()`, but they differ in caller context and entry points:

Aspect	Direct reclaim	kswapd
Caller	Allocating task (slowpath)	`kswapd` kernel thread (∼7078)
Entry	`try_to_free_pages`	`balance_pgdat` → `kswapd_shrink_node`
Shared core	`shrink_node`	`shrink_node`
GFP context	Caller's `gfp_mask`	`GFP_KERNEL`
PressurePause	Yes (hook at `do_try_to_free_pages`)	No (unless explicitly guarded)

Table 1. *Comparison of foreground direct reclaim and background `kswapd`.*

4.6 Pressure Stall Information (PSI)

PSI accounts time tasks spend stalled on memory, I/O, or CPU. For PressurePause, the **gate** uses memory PSI `some` avg10 (tasks stalled on memory); `full` is also sampled for debug peaks.

- **Types:** `PSI_MEM_SOME`, `PSI_MEM_FULL` in `include/linux/psi_types.h` (unchanged by the patch).

- **Accounting / readers:** `kernel/sched/psi.c` extends existing PSI averaging with `psi_mem_full_avg10()` and `psi_mem_some_avg10()` (`include/linux/psi.h` declares them).
- **Userspace:** `/proc/pressure/memory` (`some` vs. `full` averages)

`psi_memstall_enter()` in `__alloc_pages_direct_reclaim` marks the *current allocating task* as memory-stalled during reclaim for PSI statistics. That is distinct from `psi_mem_some_avg10()`, which the patch reads at the hook to gate bounded coordination before `shrink_zones()`.

4.7 Hook Justification

`shrink_node()` is shared by both `kswapd` and direct reclaim. Hooking only at `do_try_to_free_pages()` keeps the change on the allocating-thread path. Hooking at `shrink_node()` would require extra guards and would affect a broader surface.

Build and boot screenshots for the baseline kernel appear in Section 3; line numbers above refer to the patched v6.8.12 tree in `kernel/linux`.

5. Design and Alternative Methodologies

Chosen approach: PSI-gated bounded backoff on direct reclaim entry—gate on memory PSI `some` avg10, wake `kswapd`, wait up to `PRESSURE_PAUSE_MAX_MS`, then fall through to stock reclaim on timeout or lack of progress.

Approach	Pros	Cons
A. PSI-gated backoff (chosen)	Uses kernel stall signal; small code surface	Lock-context constraints; must validate no regressions
B. Fixed sleep in reclaim	Simple to implement	Blind to real pressure; may delay recovery
C. User-space only (swappiness, cgroups)	No kernel patch risk	Does not satisfy course kernel-modification requirement

Table 2. *Alternative methodologies considered for coordinating reclaim under thrash.*

Constants in `mm/vmscan.c`: `PSI_MEM_THRESHOLD_BPS=1` (0.01% stall, fixed-point compare via `psi_mem_avg10_exceeds_bps()`), `PRESSURE_PAUSE_MAX_MS=20`. Activation and skip counters: `/sys/kernel/debug/pressure_pause_activations`.

6. Implementation

PressurePause changes are applied in the vendored tree at `kernel/linux`. The authoritative unified diff is `kernel/patches/0001-pressurepause.patch`.

Saved config: `kernel/configs/config-6.8.12-pressurepause` (`CONFIG_PSI=y`, `LOCALVERSION="-pressurepause"`).

The superseded 0002 threshold-fix patch records an intermediate step; the report matches the final tree and `0001-pressurepause.patch`.

6.1 Stock path vs. patched path

Before (stock v6.8.12)	After (PressurePause)
`do_try_to_free_pages` → `shrink_zones` loop only	Same entry + `pressure_pause_maybe()` first
No system-wide PSI read at reclaim hook	`psi_mem_some_avg10()` / `psi_mem_full_avg10()`
No debugfs counters	`/sys/kernel/debug/pressure_pause_activations`

Table 3. *Direct-reclaim code path before and after the patch.*

6.2 Files changed

- `include/linux/psi.h` — export PSI avg10 readers and stubs when `CONFIG_PSI` is off.
- `kernel/sched/psi.c` — implement both readers using the same locking path as `psi_show()`.
- `mm/vmscan.c` — threshold helpers, `pressure_pause_maybe()`, zone/kswapd helpers, debugfs counters, and the hook call.

6.3 Full implementation (patched source)

The following excerpts match the PressurePause tree at `kernel/linux` (Linux v6.8.12 with the project patch applied). They are inlined so the report compiles in minimal uploads (e.g. Overleaf) without vendoring the full kernel. The machine-readable unified diff for `patch -p1` remains `kernel/patches/0001-pressurepause.patch`; after changing the patch, update these listings to stay in sync.

include/linux/psi.h: prototypes (lines 22–31)

```
1  void psi_memstall_enter(unsigned long *flags);
2  void psi_memstall_leave(unsigned long *flags);
3
4  unsigned long psi_mem_full_avg10(void);
5  unsigned long psi_mem_some_avg10(void);
6
7  int psi_show(struct seq_file *s, struct psi_group *group, enum psi_res res);
8  struct psi_trigger *psi_trigger_create(struct psi_group *group, char *buf,
9                         enum psi_res res, struct file *file,
```

```
10                          struct kernfs_open_file *of);
```

include/linux/psi.h: stubs when CONFIG_PSI is off (lines 51–56)

```
1  static inline void psi_init(void) {}
2
3  static inline void psi_memstall_enter(unsigned long *flags) {}
4  static inline void psi_memstall_leave(unsigned long *flags) {}
5  static inline unsigned long psi_mem_full_avg10(void) { return 0; }
6  static inline unsigned long psi_mem_some_avg10(void) { return 0; }
```

kernel/sched/psi.c: system avg10 readers (lines 1225–1254)

```
1  static unsigned long psi_mem_avg10(enum psi_states state)
2  {
3      unsigned long avg;
4      u64 now;
5
6      if (static_branch_likely(&psi_disabled))
7          return 0;
8
9      mutex_lock(&psi_system.avgs_lock);
10     now = sched_clock();
11     collect_percpu_times(&psi_system, PSI_AVGS, NULL);
12     if (now >= psi_system.avg_next_update)
13         psi_system.avg_next_update = update_averages(&psi_system, now);
14     avg = psi_system.avg[state][0];
15     mutex_unlock(&psi_system.avgs_lock);
16
17     return avg;
18 }
19
20 unsigned long psi_mem_full_avg10(void)
21 {
22     return psi_mem_avg10(PSI_MEM_FULL);
23 }
24 EXPORT_SYMBOL_GPL(psi_mem_full_avg10);
25
26 unsigned long psi_mem_some_avg10(void)
27 {
28     return psi_mem_avg10(PSI_MEM_SOME);
29 }
30 EXPORT_SYMBOL_GPL(psi_mem_some_avg10);
```

mm/vmscan.c: added include (lines 63–67)

```
1  #include <linux/swapops.h>
2  #include <linux/balloon_compaction.h>
3  #include <linux/sched/sysctl.h>
4  #include <linux/sched/loadavg.h>
5
6  #include "internal.h"
```

mm/vmscan.c: PressurePause under `#ifdef CONFIG_PSI` (lines 6173–6366)

```
#ifdef CONFIG_PSI
/*
 * PSI avg10 threshold in basis points (100 bps = 1.00% as shown in
 * /proc/pressure/memory).  Old LOAD_INT(psi) >= 25 compared the integer
 * part of the fixed-point average to 25, i.e. 25.00% stall -- not 0.25%.
 * Benchmarks peaked near full avg10=0.18 (0.18%) and never crossed that gate.
 */
#define PSI_MEM_THRESHOLD_BPS   1
#define PRESSURE_PAUSE_MAX_MS   20

/* Same fixed-point units as psi_mem_*_avg10() / /proc/pressure/memory avg10. */
static inline unsigned long psi_mem_threshold_fixed(unsigned int bps)
{
    return (unsigned long)FIXED_1 * bps / 10000;
}

static inline bool psi_mem_avg10_exceeds_bps(unsigned long avg, unsigned int bps)
{
    return avg >= psi_mem_threshold_fixed(bps);
}

static atomic64_t pressure_pause_activations;
static atomic64_t pressure_pause_entered;
static atomic64_t pressure_pause_skip_psi;
static atomic64_t pressure_pause_skip_zones_ok;
static atomic64_t pressure_pause_skip_no_reclaimable;
static atomic64_t pressure_pause_skip_cgroup;
static atomic64_t pressure_pause_peak_psi_some;
static atomic64_t pressure_pause_peak_psi_full;

static void pressure_pause_note_psi(unsigned long psi_some, unsigned long psi_full)
{
    unsigned long peak;

    peak = atomic64_read(&pressure_pause_peak_psi_some);
    if (psi_some > peak)
        atomic64_set(&pressure_pause_peak_psi_some, psi_some);

    peak = atomic64_read(&pressure_pause_peak_psi_full);
    if (psi_full > peak)
        atomic64_set(&pressure_pause_peak_psi_full, psi_full);
}

static int pressure_pause_count_show(struct seq_file *m, void *p)
{
    seq_printf(m, "activations %lld\n",
    atomic64_read(&pressure_pause_activations));
    seq_printf(m, "entered %lld\n", atomic64_read(&pressure_pause_entered));
    seq_printf(m, "skip_psi_low %lld\n",
    atomic64_read(&pressure_pause_skip_psi));
    seq_printf(m, "skip_zones_ok %lld\n",
    atomic64_read(&pressure_pause_skip_zones_ok));
```

```
50      seq_printf(m, "skip_no_reclaimable %lld\n",
51             atomic64_read(&pressure_pause_skip_no_reclaimable));
52      seq_printf(m, "skip_cgroup %lld\n",
        atomic64_read(&pressure_pause_skip_cgroup));
53      seq_printf(m, "peak_psi_some_avg10 %llu.%02llu\n",
54             LOAD_INT(atomic64_read(&pressure_pause_peak_psi_some)),
55             LOAD_FRAC(atomic64_read(&pressure_pause_peak_psi_some)));
56      seq_printf(m, "peak_psi_full_avg10 %llu.%02llu\n",
57             LOAD_INT(atomic64_read(&pressure_pause_peak_psi_full)),
58             LOAD_FRAC(atomic64_read(&pressure_pause_peak_psi_full)));
59      return 0;
60  }
61  DEFINE_SHOW_ATTRIBUTE(pressure_pause_count);
62
63  static int __init pressure_pause_debugfs_init(void)
64  {
65      debugfs_create_file("pressure_pause_activations", 0444, NULL, NULL,
66                  &pressure_pause_count_fops);
67      return 0;
68  }
69  late_initcall(pressure_pause_debugfs_init);
70
71  /* True if any managed zone in the zonelist still needs reclaim (not at min).
        */
72  static bool pressure_pause_under_pressure(struct zonelist *zonelist,
73                        struct scan_control *sc)
74  {
75      struct zoneref *z;
76      struct zone *zone;
77      bool any = false;
78
79      for_each_zone_zonelist_nodemask(zone, z, zonelist, sc->reclaim_idx,
80                      sc->nodemask) {
81          if (!managed_zone(zone))
82              continue;
83          any = true;
84          if (!zone_watermark_ok(zone, sc->order, min_wmark_pages(zone),
85                         sc->reclaim_idx, 0))
86              return true;
87      }
88      return false;
89  }
90
91  /* True if any zone has reached min watermarks (progress / relief). */
92  static bool pressure_pause_any_zone_ok(struct zonelist *zonelist,
93                         struct scan_control *sc)
94  {
95      struct zoneref *z;
96      struct zone *zone;
97
98      for_each_zone_zonelist_nodemask(zone, z, zonelist, sc->reclaim_idx,
99                      sc->nodemask) {
100         if (!managed_zone(zone))
101             continue;
102         if (zone_watermark_ok(zone, sc->order, min_wmark_pages(zone),
103                         sc->reclaim_idx, 0))
```

```
104             return true;
105     }
106     return false;
107 }
108
109 static bool pressure_pause_has_reclaimable(struct zonelist *zonelist,
110                         struct scan_control *sc)
111 {
112     struct zoneref *z;
113     struct zone *zone;
114
115     for_each_zone_zonelist_nodemask(zone, z, zonelist, sc->reclaim_idx,
116                     sc->nodemask) {
117         if (!managed_zone(zone))
118             continue;
119         if (zone_reclaimable_pages(zone))
120             return true;
121     }
122     return false;
123 }
124
125 static void pressure_pause_wake_kswapd(struct zonelist *zonelist,
126                         struct scan_control *sc)
127 {
128     struct zoneref *z;
129     struct zone *zone;
130
131     for_each_zone_zonelist_nodemask(zone, z, zonelist, sc->reclaim_idx,
132                     sc->nodemask) {
133         if (!managed_zone(zone))
134             continue;
135         wakeup_kswapd(zone, sc->gfp_mask, sc->order, sc->reclaim_idx);
136     }
137 }
138
139 /*
140  * PSI-gated bounded coordination before synchronous shrink_zones().
141  * Falls through to stock reclaim on timeout or lack of progress.
142  */
143 static void pressure_pause_maybe(struct zonelist *zonelist,
144                  struct scan_control *sc)
145 {
146     unsigned long psi_full, psi_some;
147     unsigned long deadline;
148
149     atomic64_inc(&pressure_pause_entered);
150
151     if (cgroup_reclaim(sc)) {
152         atomic64_inc(&pressure_pause_skip_cgroup);
153         return;
154     }
155
156     psi_full = psi_mem_full_avg10();
157     psi_some = psi_mem_some_avg10();
158     pressure_pause_note_psi(psi_some, psi_full);
159
```

```
160     /* Gate on memory PSI "some" -- rises earlier than "full" under swap
        thrash. */
161     if (!psi_mem_avg10_exceeds_bps(psi_some, PSI_MEM_THRESHOLD_BPS)) {
162         atomic64_inc(&pressure_pause_skip_psi);
163         return;
164     }
165
166     if (!pressure_pause_has_reclaimable(zonelist, sc)) {
167         atomic64_inc(&pressure_pause_skip_no_reclaimable);
168         return;
169     }
170
171     /* Only skip if no zone in the zonelist is under min watermarks. */
172     if (!pressure_pause_under_pressure(zonelist, sc)) {
173         atomic64_inc(&pressure_pause_skip_zones_ok);
174         return;
175     }
176
177     pressure_pause_wake_kswapd(zonelist, sc);
178     atomic64_inc(&pressure_pause_activations);
179
180     deadline = jiffies + msecs_to_jiffies(PRESSURE_PAUSE_MAX_MS);
181     while (time_before(jiffies, deadline)) {
182         if (fatal_signal_pending(current))
183             break;
184         if (pressure_pause_any_zone_ok(zonelist, sc))
185             break;
186         schedule_timeout_idle(1);
187     }
188 }
189 #else
190 static void pressure_pause_maybe(struct zonelist *zonelist,
191                  struct scan_control *sc)
192 {
193 }
194 #endif
```

mm/vmscan.c: hook in `do_try_to_free_pages` (lines 6394–6397)

```
1     if (!cgroup_reclaim(sc))
2         __count_zid_vm_events(ALLOCSTALL, sc->reclaim_idx, 1);
3
4     pressure_pause_maybe(zonelist, sc);
```

6.4 `pressure_pause_maybe()` control flow

On each `do_try_to_free_pages()` entry the hook increments `entered`, then:

1. **Cgroup reclaim** → increment `skip_cgroup`, return (stock reclaim only).
2. Read `psi_mem_some_avg10()` and `psi_mem_full_avg10()`; update `peak_psi_*` maxima.
3. If some avg10 is below 0.01% (`PSI_MEM_THRESHOLD_BPS=1`) → `skip_psi_low`, return.

4. If no zone has reclaimable pages → `skip_no_reclaimable`, return.
5. If *no* zone in the zonelist is below min watermarks → `skip_zones_ok`, return. (Stock logic incorrectly skipped when *any* zone was OK; the fix requires at least one zone still under pressure.)
6. Wake `kswapd` per managed zone; increment `activations`; bounded wait up to `PRESSURE_PAUSE_MAX_MS` (20 ms), exiting early on fatal signal or if any zone reaches min watermarks.
7. Always fall through: the stock `shrink_zones()` priority loop runs unchanged.

6.5 PSI threshold change

The first implementation used `LOAD_INT(psi_full) >= 25`, which requires **25%** memory `full` stall (the integer part of the fixed-point average matches the whole number in `/proc/pressure/memory`, e.g. `avg10=25.00`). Benchmarks peaked near `full avg10=0.18` (0.18%), so the gate never fired and `activations` stayed at zero.

The fix compares the same fixed-point avg10 as `/proc` using basis points:

mm/vmscan.c: threshold compare and gate (lines 6189–6192, 6332–6336)

```
1  static inline bool psi_mem_avg10_exceeds_bps(unsigned long avg, unsigned int
       bps)
2  {
3      return avg >= psi_mem_threshold_fixed(bps);
4  }
5  ...
6      if (!psi_mem_avg10_exceeds_bps(psi_some, PSI_MEM_THRESHOLD_BPS)) {
7          atomic64_inc(&pressure_pause_skip_psi);
8          return;
9      }
```

The gate uses memory PSI `some` avg10, which rises earlier than `full` under swap-heavy thrash.

6.6 Debugfs interface

Mount debugfs if needed (`sudo mount -t debugfs none /sys/kernel/debug`), then read `/sys/kernel/debug/pressure_pause_activations`:

Field	Meaning
`activations`	Coordination ran (wake `kswapd` + bounded wait)
`entered`	Hook invoked from `do_try_to_free_pages`
`skip_psi_low`	`some` avg10 below 0.01% threshold
`skip_zones_ok`	No zone below min watermarks
`skip_no_reclaimable`	No reclaimable pages in zonelist
`skip_cgroup`	Cgroup reclaim path
`peak_psi_some_avg10`	Max `some` avg10 seen at hook since boot
`peak_psi_full_avg10`	Max `full` avg10 seen at hook since boot

Table 4. *PressurePause debugfs counters (`mm/vmscan.c`).*

6.7 Debugfs verification (screenshots)

The following pair shows `/sys/kernel/debug/pressure_pause_activations` (and related shell output) **before** wiring the full multi-field counter file vs **after** booting the final PressurePause kernel and exercising memory pressure so counters and peaks populate.

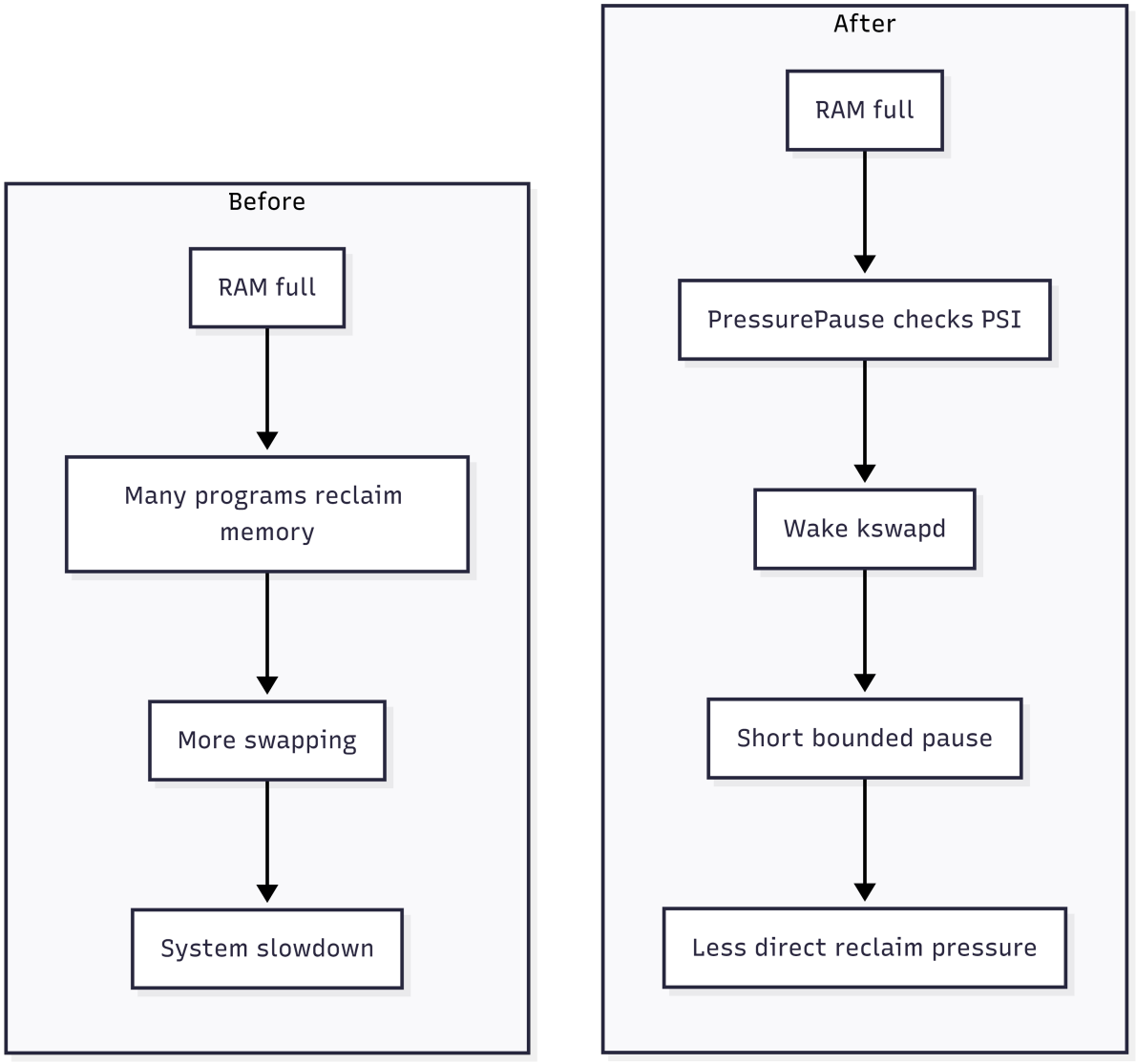


Figure 11. ***Left:*** *debugfs verification before the final counter format / activation behavior.* ***Right:*** *same interface after the patch under load, showing non-zero* `entered`*,* `activations`*, and skip / peak PSI fields as implemented in* `mm/vmscan.c`*.*

6.8 Build and install

Build patched kernel

```
cd kernel/linux
scripts/config --enable PSI
scripts/config --set-str LOCALVERSION "-pressurepause"
make olddefconfig && make -j$(nproc)
cd ../.. && ./scripts/install-pressurepause-kernel.sh
```

Boot verification:

- Boot through GRUB → Advanced options, then choose `6.8.12-pressurepause` (or `6.8.12-pressurepause+` when the parent repo has local edits).
- Confirm `uname -r`. Then read `/proc/pressure/memory` and run `./scripts/verify-activation.sh`.
- The patched kernel booted successfully; repeated 120 s `stress-ng` workloads completed without crash or hang, and debugfs `activations` increased during benchmark runs (Section 8).

7. Correctness and Safety

1. **Bounded wait:** the pause is capped at 20 ms and uses one-tick idle sleep slices.
2. **Never skip reclaim:** the hook always returns to the stock `shrink_zones()` priority loop.
3. **kswapd cooperation:** `wakeup_kswapd()` is additive; foreground reclaim still proceeds.
4. **Livelock guard:** skip pause if `zone_reclaimable_pages()` is zero for all zones in the zonelist.
5. **Fallback:** Ubuntu `6.8.0-117-generic` and `6.8.12-baseline` remain in GRUB if the patched kernel panics.

8. Evaluation

This section records evidence that the PressurePause code path runs under load; the primary implementation narrative is in Section 6.

8.1 Methodology

Matched pairs on the same host used `./bench-run.sh` with 120 s `stress-ng` memory workloads, `vmstat`, `pgmajfault`, PSI samples, and debugfs activation snapshots (`pressure-pause-debugfs-*.txt`).

Representative directories:

- **8 workers / 93%:** baseline `bench-6.8.12-baseline-20260519-135148` patched `bench-6.8.12-pressurepause+-20260519-155900`.
- **4 workers / 85%:** baseline `bench-6.8.12-baseline-20260519-135638` patched runs `160116`, `160400`, and `160617`.

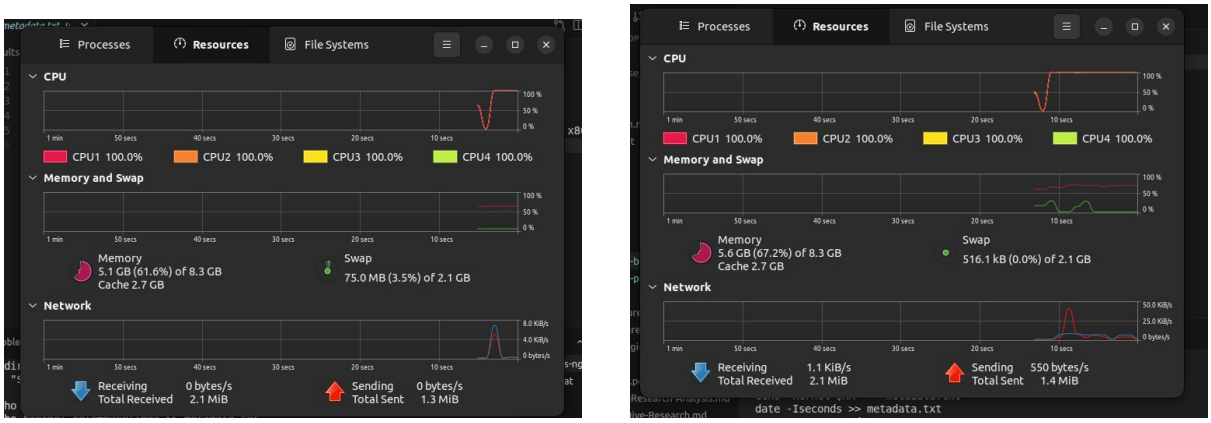


Figure 12. ***Left:*** *representative benchmark / monitoring session on the* ***baseline*** *kernel during a* `stress-ng` *memory workload.* ***Right:*** *comparable session on the* ***PressurePause*** *kernel (same host class of workload; 120 s timeouts and tool output as in* `./bench-run.sh`*).*

Workload	Metric	Baseline	PressurePause+
8 vm / 93%	Avg swap-out (KB/s)	31222.5	25638.6
8 vm / 93%	End swap (KB)	37992	25952
8 vm / 93%	`activations` Δ	–	+40
8 vm / 93%	`pgmajfault` Δ	222	321
4 vm / 85%	Avg swap-out (KB/s)	33380.0	22791.8
4 vm / 85%	Max swapped (KB)	1215952	913977.3
4 vm / 85%	End swap (KB)	91676	26104
4 vm / 85%	`activations` Δ	–	+15 total
4 vm / 85%	`pgmajfault` Δ	5	112.7

Table 5. *Matched benchmark summary (May 2026). Patched runs used the rebuilt PressurePause+ kernel with debugfs activation counters. The 4 vm / 85% patched column averages three patched trials.*

8.2 Discussion

Kernels built with the original `LOAD_INT(psi_full) >= 25` gate could complete stress workloads with **zero** `activations` because observed `full` avg10 never reached 25%. After the basis-point gate on `some` avg10 (Section 6.3), debugfs confirms the hook runs: on the 8 vm / 93% workload, `activations` rose from 15 to 55 (+40) during the run; on one 4 vm / 85% trial (160617), +15 activations. Matched baseline vs patched runs also show lower swap-out and residual swap on the patched kernel, with higher `pgmajfault` as a documented tradeoff (average swap-out −17.9% on 8 vm / 93%, −31.7% on the 4 vm / 85% patched average).

8.3 Limitations

This is not a full statistical study. PSI samples are every 5 s and can miss short spikes seen at the reclaim hook, which is why the debugfs activation counter is more important than sampled `avg10` alone. Baseline PSI still reports "no PSI," so baseline and patched PSI totals cannot be compared directly. Major-fault counts rise on the patched kernel; future work should add repeated trials, latency percentiles, and a workload sweep to find where the swap reduction is worth the fault-cost tradeoff.

9. Conclusion

PressurePause adds `pressure_pause_maybe()` on the direct-reclaim path: when memory PSI `some` avg10 exceeds 0.01%, the allocating thread wakes `kswapd`, waits up to 20 ms, then always proceeds with stock `shrink_zones()`. Supporting changes export system PSI averages, fix the zone watermark guard, and expose debugfs counters for verification. Under `stress-ng`, the patched kernel is stable and debugfs shows real activations after the threshold fix; matched benchmarks report lower swap pressure at the cost of higher major page faults. The project delivers a small, PSI-gated reclaim coordination patch with observable behavior and measured tradeoffs.